PROJECT

Functional Architecture of a Level 4 Autonomous Vehicle

Design of the perception, decision and control modules for a geofenced urban robotaxi, with a software prototype

---

Submitted by

Name	Prajakta Ghosh
Enrollment No.	12023002002107
Registration No.	304202300900494
Department	Electronics and Communication Engineering (ECE)
Section	4B
Roll No.	34

September 2026

Table of Contents

Abstract....3
1. Introduction....3
1.1 Problem statement....3
1.2 Objectives....3
1.3 Methodology....4
2. Use Case and Operational Design Domain....4
3. Requirements....4
3.1 Functional requirements....4
3.2 Performance requirements....5
3.3 Safety goals....5
4. Overall Functional Architecture....6
4.1 Design principles....6
4.2 Module interfaces....7
5. Sensor and Hardware Architecture....7
5.1 Sensor suite....7
5.2 Computing, network and power....8
6. Perception Module....9
6.1 Pre-processing, synchronisation and calibration....10
6.2 Object detection....10
6.3 Sensor fusion....11
6.4 Multi-object tracking....11
6.5 Occupancy grid and free space....11
6.6 Localization....11
6.7 World model output....11
7. Decision Module....12
7.1 Motion prediction....12
7.2 Mission (route) planning....12
7.3 Behaviour planning....12
7.4 Motion (trajectory) planning....13
7.5 Trajectory safety checker....14
8. Control Module....14
8.1 Vehicle model....15
8.2 Lateral control....15
8.3 Longitudinal control....15
8.4 Actuator interface....15
9. Safety and Fallback Architecture....16
9.1 Redundancy and fault detection....16
9.2 Degradation modes and minimal risk manoeuvre....16
9.3 Remote assistance....17
9.4 SOTIF and cybersecurity....17
10. Software Architecture and Timing....17
11. Prototype Simulation and Results....18
11.1 Prototype description....18
11.2 Scenario A: nominal mission....18
11.3 Scenario B: perception failure....19
11.4 Results....20
11.5 How to run....21
12. Verification and Validation Strategy....21

13. Conclusion and Future Work..... 21
References..... 22

Abstract

This project designs the functional architecture of a Level 4 (High Driving Automation) vehicle as defined by SAE J3016: a vehicle that performs the whole driving task and its own fallback inside a defined operational design domain (ODD), with no human driver required. The chosen use case is an electric robotaxi serving a geofenced urban district at speeds up to 50 km/h. Starting from the ODD and a set of functional, performance and safety requirements, the report specifies the sensor suite and computing hardware, and then details the three core software modules: **perception** (multi-sensor detection, bird's-eye-view fusion, tracking and localization), **decision** (prediction, route, behaviour and motion planning with a safety envelope) and **control** (model predictive lateral control and feed-forward longitudinal control over drive-by-wire actuators). A safety architecture with an independent safety computer, degradation modes and a minimal risk manoeuvre ensures that the vehicle can reach a safe stop after any single failure. The design is supported by seven architecture diagrams, interface and timing tables, and a working Python prototype that runs the perception, behaviour-planning and control loop in closed loop on two scenarios: a nominal trip with a lead vehicle, a red light, a crossing pedestrian and a drop-off, and a perception-failure case that triggers a minimal risk manoeuvre.

1. Introduction

1.1 Problem statement

A Level 4 vehicle must do everything a human driver does, and must also handle its own failures, because nobody on board is expected to take over. This changes the architecture fundamentally compared with a Level 2 driver-assistance system: the system must build a complete model of its surroundings, make every tactical decision, control the vehicle precisely, and continuously check itself so that it can stop safely when something goes wrong. The task of this project is to design such an architecture at the functional level, that is, to define which functions exist, what each one does, how they exchange data, and how the whole system stays safe.

1.2 Objectives

1. Define the use case, ODD and requirements of a Level 4 vehicle.
2. Design the overall functional architecture and the sensor, computing and actuation hardware it runs on.
3. Detail the **perception**, **decision** and **control** modules, including algorithms, interfaces and update rates.
4. Design the safety and fallback architecture (redundancy, degradation modes, minimal risk manoeuvre).
5. Validate the data flow and decision logic with a closed-loop software prototype.

1.3 Methodology

The design follows a top-down systems-engineering approach commonly used in the automotive industry (the left side of the V-model): use case and ODD, then requirements and safety goals, then functional architecture, then module design, then verification. Design choices are based on published architectures of open-source stacks (Autoware, Baidu Apollo) and on the documented sensor suites of deployed Level 4 services such as Waymo's 6th-generation Driver.

2. Use Case and Operational Design Domain

The vehicle is an electric four-seat robotaxi that carries passengers between pick-up and drop-off points within a mapped urban district, for example a planned township or IT park with a regular road grid. Passengers book rides through an app; the vehicle drives itself with no safety driver, supported by a remote assistance centre that can give guidance but never drives the vehicle remotely. Table 1 defines the ODD. Outside it, the vehicle will not start a trip, and if it is about to leave the ODD during a trip it performs a minimal risk manoeuvre.

Table 1: Operational Design Domain of the proposed Level 4 robotaxi

ODD attribute	Inside the ODD	Outside the ODD (triggers refusal or MRM)
Geography	Geofenced, HD-mapped district of about 25 km²	Unmapped roads, map older than validity period
Road types	Urban arterials and local streets, 1 to 3 lanes per direction, signalised and unsignalised junctions, roundabouts	Expressways, unpaved roads, construction zones without map update
Speed	0 to 50 km/h	Above 50 km/h
Traffic participants	Cars, buses, two- and three-wheelers, cyclists, pedestrians, animals at low speed	Police manual traffic control (handled with remote assistance)
Weather	Dry, light to moderate rain (up to about 10 mm/h), light fog (visibility above 150 m)	Heavy rain, flooding, dense fog, dust storm
Lighting	Day and night with street lighting	Sensor blinding that persists beyond cleaning cycle
Connectivity	4G/5G for remote assistance (driving does not depend on it)	Loss of connectivity alone does not end the ODD but restricts new trips

3. Requirements

3.1 Functional requirements

Table 2: Functional requirements

ID	Requirement
FR-01	The vehicle shall drive from any pick-up point to any drop-off point inside the ODD without human input.

ID	Requirement
FR-02	The vehicle shall detect, classify and track all road users and obstacles within 360° around the vehicle.
FR-03	The vehicle shall determine its position on the HD map with lateral error below 10 cm and heading error below 0.3°.
FR-04	The vehicle shall obey traffic signals, signs, lane markings, speed limits and right-of-way rules.
FR-05	The vehicle shall yield to pedestrians and cyclists and keep safe distances to all road users.
FR-06	The vehicle shall perform lane changes, turns, merges and nudges around static obstacles when safe.
FR-07	The vehicle shall monitor its own health and the ODD, and perform a minimal risk manoeuvre when it cannot continue safely.
FR-08	The vehicle shall give way to emergency vehicles detected visually or acoustically.
FR-09	The vehicle shall communicate with passengers and the remote assistance centre and accept high-level guidance.
FR-10	The vehicle shall record data needed for incident analysis (DSSAD / event data recorder).

3.2 Performance requirements

Table 3: Performance requirements

Parameter	Target	Reasoning
Vehicle detection range	≥ 150 m (front), ≥ 80 m (sides, rear)	Closing speed of up to 100 km/h at junctions and oncoming traffic
Pedestrian detection range	≥ 80 m	Stopping distance at 50 km/h is about 30 m at 4 m/s², leaving margin for classification and prediction
Sensor-to-command latency	≤ 150 ms (99th percentile)	Adds only about 2 m of travel at 50 km/h
Localization accuracy	≤ 10 cm lateral, ≤ 20 cm longitudinal	Lane widths of about 3.2 to 3.5 m in the ODD
Lateral tracking error	≤ 20 cm (straight), ≤ 30 cm (curves)	Stay within lane with margin for wide vehicles
Comfort limits	$\|a\| \le 2$ m/s² nominal, jerk ≤ 2.5 m/s³	Passenger comfort; emergency braking up to 8 m/s² allowed
Time to minimal risk condition	≤ 10 s after critical fault	Limits exposure time with degraded capability

3.3 Safety goals

A hazard analysis and risk assessment (HARA) according to ISO 26262 would derive the safety goals and their Automotive Safety Integrity Levels (ASIL). The following indicative safety goals drive the architecture:

Table 4: Indicative safety goals

ID	Safety goal	ASIL
SG-1	Avoid unintended or excessive steering that leads to leaving the lane or collision.	D
SG-2	Avoid unintended acceleration and loss of braking capability.	D
SG-3	Avoid collisions caused by missed, late or wrongly classified objects (also addressed by ISO 21448 SOTIF).	D
SG-4	Reach a minimal risk condition after any single critical fault.	D
SG-5	Prevent automated operation outside the ODD.	B

4. Overall Functional Architecture

Figure 1 shows the proposed architecture. It follows the classical **sense, perceive, decide, act** pipeline, with five main layers connected by well-defined interfaces, and four cross-cutting services beneath them. Data flows from left to right: raw sensor data is turned into a world model, the world model into a trajectory, and the trajectory into actuator commands. Feedback from the vehicle closes the control loop, and the safety supervisor watches every layer.

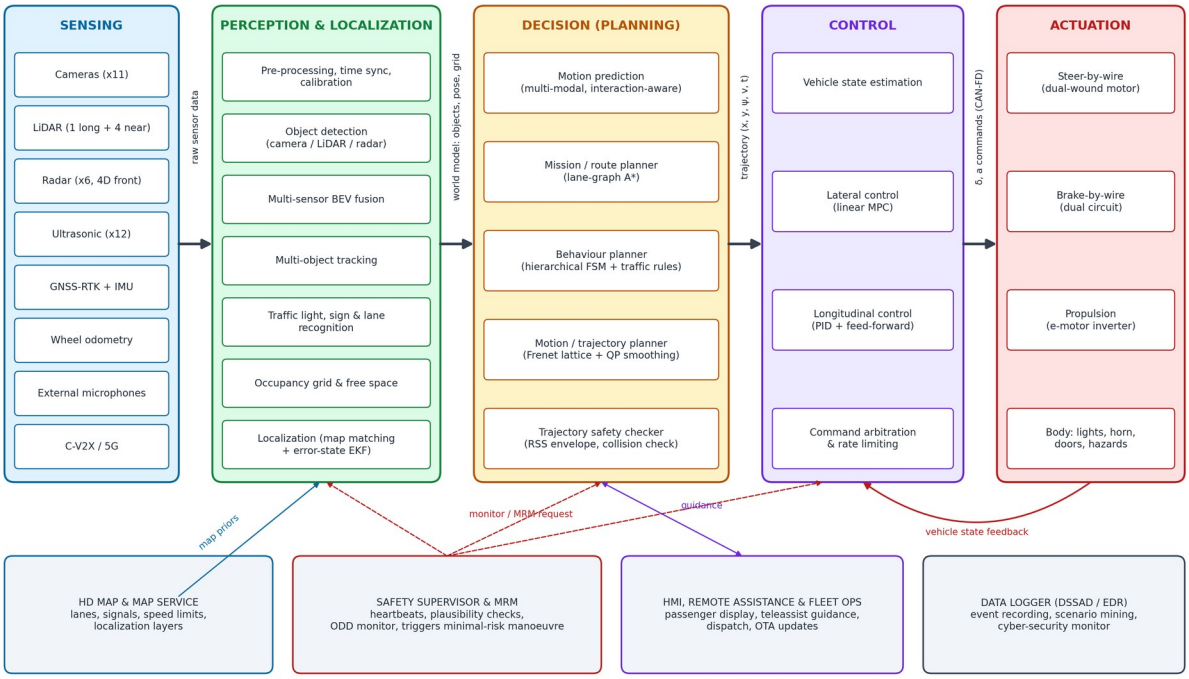


Figure 1: Functional architecture of the proposed Level 4 vehicle

4.1 Design principles

- **Modularity**: each function is a separate software component with a typed interface, so that it can be developed, tested and certified independently and replaced (for example a new detection network) without changing the rest.
- **Diversity and redundancy**: three sensing modalities (camera, lidar, radar), two computers, two steering and braking channels and two power networks, so that no single fault removes the ability to stop safely.

- **Doer / checker**: complex, AI-based functions (the doer) are checked by simpler, verifiable functions (the checker), such as the rule-based trajectory safety checker and the independent safety computer.
- **Graceful degradation**: faults lead to a reduced but safe level of service rather than an abrupt loss of function.
- **Determinism and time synchronisation**: all data carries a common timestamp (PTP, IEEE 802.1AS) and critical paths run on a real-time scheduler.

4.2 Module interfaces

Table 5: Interfaces between the main modules

Module	Main inputs	Main outputs	Rate
Sensing	Physical environment	Images, point clouds, radar detections, IMU/GNSS, wheel speeds, audio	10 to 100 Hz
Perception	Sensor data, calibration, HD map	Object tracks, occupancy grid, lanes, traffic-light states	10 Hz
Localization	IMU, GNSS, wheel odometry, lidar, HD map	Ego pose, velocity and covariance	100 Hz
Prediction	Object tracks, map, ego pose	Multi-modal future trajectories with probabilities (8 s horizon)	10 Hz
Mission planner	Trip request, lane graph, traffic data	Lane-level route	On request / 1 Hz
Behaviour planner	World model, predictions, route, traffic rules	Driving mode, target lane, speed limit, stop points	10 Hz
Motion planner	Behaviour decision, world model, predictions	Trajectory: (x, y, ψ, v, a, t) for 8 s	10 Hz
Safety checker	Trajectory, world model	Approved trajectory or fallback request	10 Hz
Control	Approved trajectory, vehicle state	Steering angle, acceleration / brake demand	100 Hz
Safety supervisor	Heartbeats and diagnostics of all modules, ODD status	System mode (nominal, degraded, MRM)	100 Hz

5. Sensor and Hardware Architecture

5.1 Sensor suite

The sensor suite (Table 7, Figure 2) is sized so that every point around the vehicle is covered by at least two different sensing modalities, and the front sector by all three. It follows the same philosophy as current commercial Level 4 vehicles; for comparison, Waymo's 6th-generation Driver uses 13 cameras, 4 lidars and 6 radars plus external audio receivers.

Table 6: Proposed sensor suite

Sensor	Qty	Placement	Key specification	Purpose
Long-range lidar	1	Roof centre	360° × 30°, 128 channels, ~200 m at 10 % reflectivity, 10-20 Hz	Primary 3D detection, map-based localization

Sensor	Qty	Placement	Key specification	Purpose
Near-field lidar	4	Body corners	Wide vertical FoV, 0.1-30 m	Blind-spot coverage, kerbs, close pedestrians
Cameras	11	3 front (tele, main, wide), 4 side, 2 rear-side, 1 rear, 1 bumper	8 MP, HDR, LED-flicker mitigation, 30 fps	Classification, traffic lights, signs, lanes
Radar	6	1 front 4D imaging, 4 corners, 1 rear	77 GHz, front up to ~300 m, direct Doppler velocity	All-weather detection and velocity
Ultrasonic	12	Bumpers and sides	0.2-5 m	Low-speed manoeuvring, parking, drop-off
GNSS-RTK	2 antennas	Roof	cm-level with RTK corrections, dual-antenna heading	Absolute position, map initialisation
IMU	2	Primary and safety computer	Automotive / tactical grade 6-axis	Motion estimation, dead reckoning
Wheel encoders	4	Wheel hubs	From ABS system	Odometry, slip detection
Microphones	4	Roof edges	Array for direction finding	Emergency-vehicle siren detection
C-V2X / 5G	1	Telematics unit	PC5 sidelink + Uu cellular	Signal phase data (SPaT), remote assistance

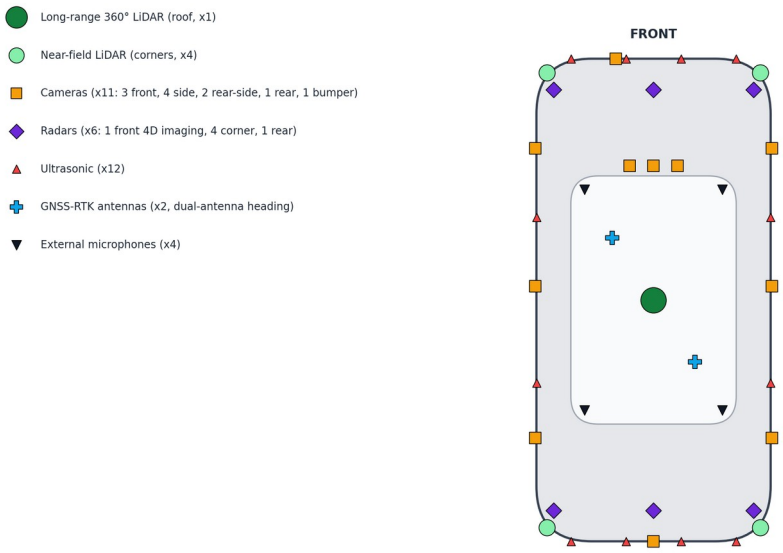


Figure 2: Placement of sensors on the proposed vehicle (top view)

5.2 Computing, network and power

Figure 3 shows the hardware topology. The **primary ADS computer** contains two high-performance automotive AI system-on-chips (in the class of NVIDIA DRIVE Orin or Thor) and runs the full perception, prediction, planning and control stack. The **safety computer** is

independent: it has its own ASIL D microcontroller, a mid-range SoC, its own IMU and a subset of sensors (front radar, front camera and the near-field lidars). It checks the primary computer's outputs and, if the primary fails, computes and executes a minimal risk manoeuvre on its own.

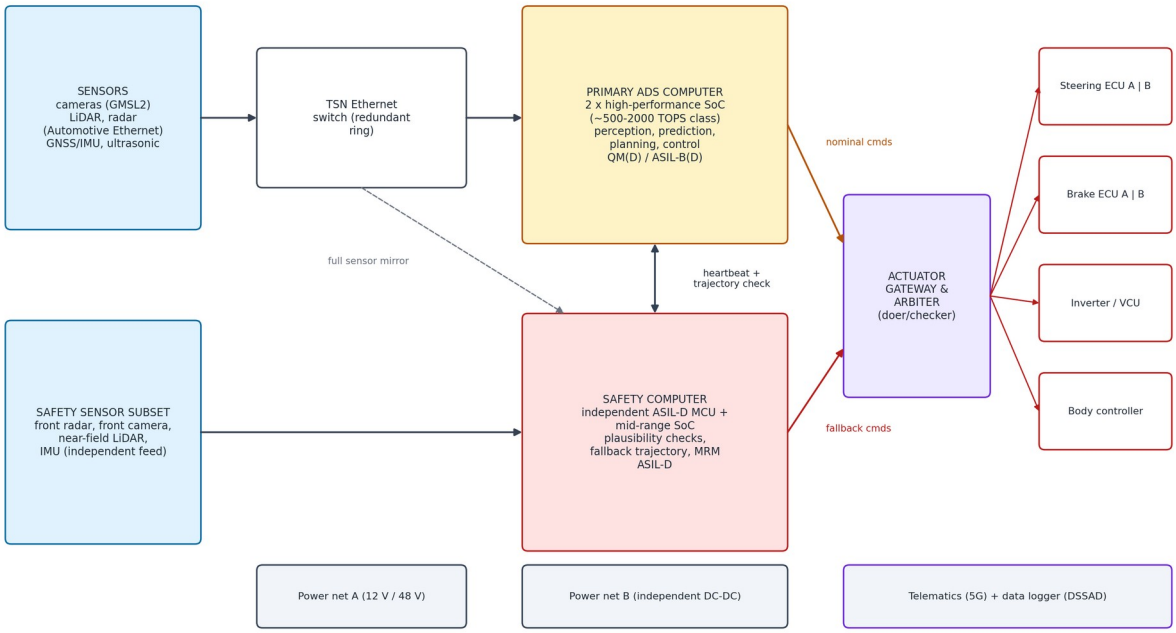


Figure 3: Computing, network and redundancy topology

- **Network**: cameras connect by GMSL2 serial links; lidars and radars by automotive Ethernet into a time-sensitive networking (TSN) switch arranged as a redundant ring; actuator ECUs by CAN-FD.
- **Actuator gateway**: arbitrates between nominal commands from the primary computer and fallback commands from the safety computer (doer / checker).
- **Actuators**: steer-by-wire with a dual-wound motor and two ECUs; brake-by-wire with two independent circuits; electric propulsion through the vehicle control unit; an electric parking brake to hold the MRC.
- **Power**: two independent low-voltage networks fed by separate DC-DC converters from the traction battery, each able to supply the safety computer and one steering and braking channel.

6. Perception Module

The perception module turns raw sensor data into the **world model**: a consistent description of everything around the vehicle and of the vehicle's own position. Figure 4 shows its internal pipeline. It is the most computation-intensive part of the stack and uses deep neural networks for detection, with classical estimation methods (Kalman filters, scan matching) for tracking and localization.

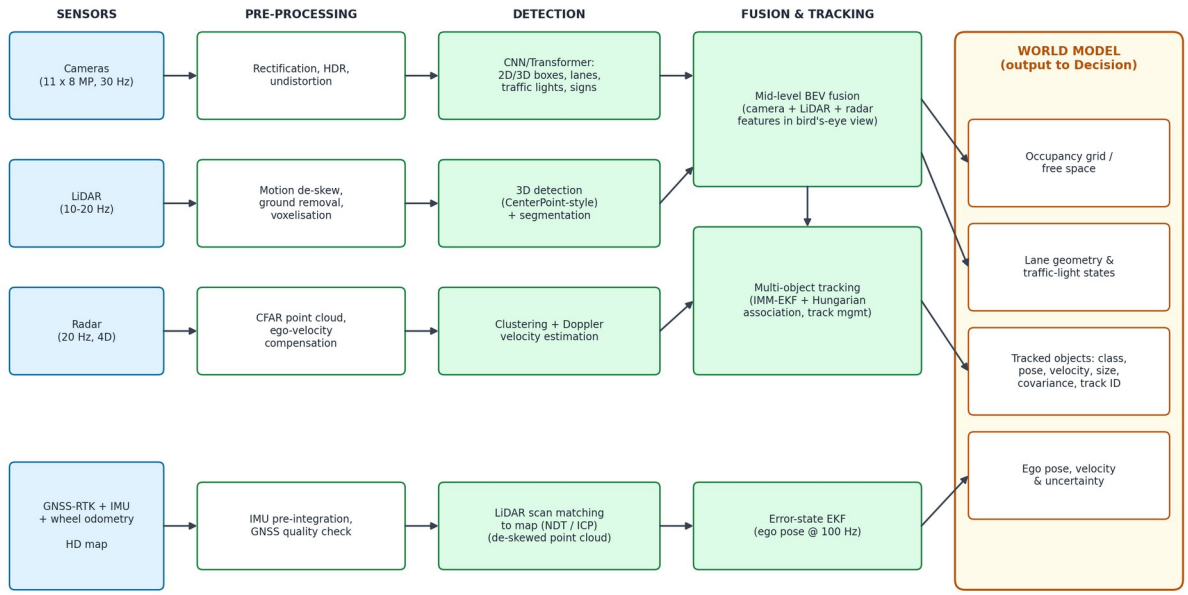


Figure 4: Perception and localization pipeline

6.1 Pre-processing, synchronisation and calibration

- All sensors are synchronised to a common clock using PTP; each frame is timestamped at capture.
- Camera images are rectified and undistorted using intrinsic calibration; lidar sweeps are de-skewed to compensate for vehicle motion during the sweep; radar detections are compensated for ego velocity.
- Extrinsic calibration (the position and orientation of every sensor relative to the vehicle frame) is done at the factory and monitored online; a calibration drift is reported to the safety supervisor.

6.2 Object detection

Table 7: Detection methods per sensor

Modality	Method	Output
Camera	Convolutional / transformer network with shared backbone and multiple heads (e.g. BEV-based 3D detection)	2D and 3D boxes with class, lane lines, drivable area, traffic-light state, traffic signs
LiDAR	Voxel- or pillar-based 3D detector (CenterPoint-style) plus point-wise segmentation	3D boxes with heading, ground and kerb segmentation
Radar	CFAR detection, DBSCAN clustering, Doppler velocity	Clusters with range, azimuth, elevation (4D) and radial velocity
Microphones	Spectrogram classifier + time-difference-of-arrival	Siren presence and direction

6.3 Sensor fusion

Three levels of fusion are possible. **Early fusion** combines raw data (for example projecting lidar points into images), which uses all information but is sensitive to calibration errors. **Late fusion** combines the object lists produced by each sensor, which is robust and simple but loses information. **Mid-level fusion** combines learned features in a shared bird's-eye-view (BEV) grid. This design uses mid-level BEV fusion as the main path, because it gives the best accuracy in current research and production systems, and adds a late, track-level fusion with radar and near-field lidar objects as a diverse second path that keeps working if a network fails.

6.4 Multi-object tracking

Fused detections are associated with existing tracks using the Hungarian algorithm on a cost combining distance, size and class, and each track is updated with an Interacting Multiple Model (IMM) extended Kalman filter that mixes a constant-velocity and a constant-turn-rate model. The standard Kalman prediction and update steps are:

$$\hat{x}_{k|k-1} = F\hat{x}_{k-1}, \qquad P_{k|k-1} = FP_{k-1}F^T + Q \tag{1}$$

$$K = PH^T(HPH^T + R)^{-1}, \quad \hat{x}_k = \hat{x}_{k|k-1} + K(z_k - H\hat{x}_{k|k-1}), \quad P_k = (I - KH)P_{k|k-1} \tag{2}$$

Tracks carry an ID, class, position, velocity, heading, size and covariance. Tracks are confirmed after several consistent detections and deleted after several misses, which suppresses false positives and bridges short occlusions.

6.5 Occupancy grid and free space

Objects that do not fit any known class (debris, fallen branches, potholes) are captured in a 2D occupancy grid around the vehicle, updated with the log-odds rule so that evidence from many scans accumulates:

$$l_t(m_i) = l_{t-1}(m_i) + \log\frac{p(m_i \mid z_t)}{1 - p(m_i \mid z_t)} - l_0 \tag{3}$$

6.6 Localization

The vehicle's pose is estimated by an error-state extended Kalman filter running at 100 Hz. The IMU drives the prediction step; corrections come from wheel odometry, RTK-GNSS when available, and lidar scan matching (Normal Distributions Transform) against the HD map's localization layer. Because scan matching does not depend on satellites, localization stays accurate under flyovers and between tall buildings. If the estimated covariance exceeds the requirement of FR-03, the supervisor switches to degraded mode.

6.7 World model output

- **Object list**: tracks with class, pose, velocity, size, covariance and ID.
- **Occupancy grid** and drivable free space.
- **Road semantics**: lane geometry (fused with the HD map), traffic-light states, temporary signs.
- **Ego state**: pose, velocity, acceleration and their uncertainty.

- **Perception health**: per-sensor status, blockage and calibration flags for the safety supervisor.

7. Decision Module

The decision module answers three questions at different time scales: where to go (mission planning, minutes), what to do (behaviour planning, seconds), and exactly how to move (motion planning, the next 8 seconds). Before planning, it predicts what other road users will do.

7.1 Motion prediction

For each tracked agent the prediction function outputs several possible future trajectories (for example going straight, turning, or stopping) with a probability for each, over an 8-second horizon. A graph or transformer network encodes the agent's history, the lane graph and the interactions between agents (VectorNet or scene-transformer style). A physics-based constant-velocity prediction runs in parallel as a fallback and as a sanity check for the learned model.

7.2 Mission (route) planning

The HD map is represented as a directed lane graph. The route from the current position to the destination is found with the A* algorithm, where the cost g(n) combines length, expected travel time, number of lane changes and turn difficulty, and the heuristic h(n) is the straight-line travel time to the goal:

$$f(n) = g(n) + h(n) \tag{4}$$

7.3 Behaviour planning

Behaviour planning uses a hierarchical finite state machine (FSM), shown in Figure 5, combined with a rule book of traffic laws. Each state defines the driving intent and passes constraints (target lane, speed limit, stop points, following target) to the motion planner. The transition from any state to the minimal risk manoeuvre has the highest priority.

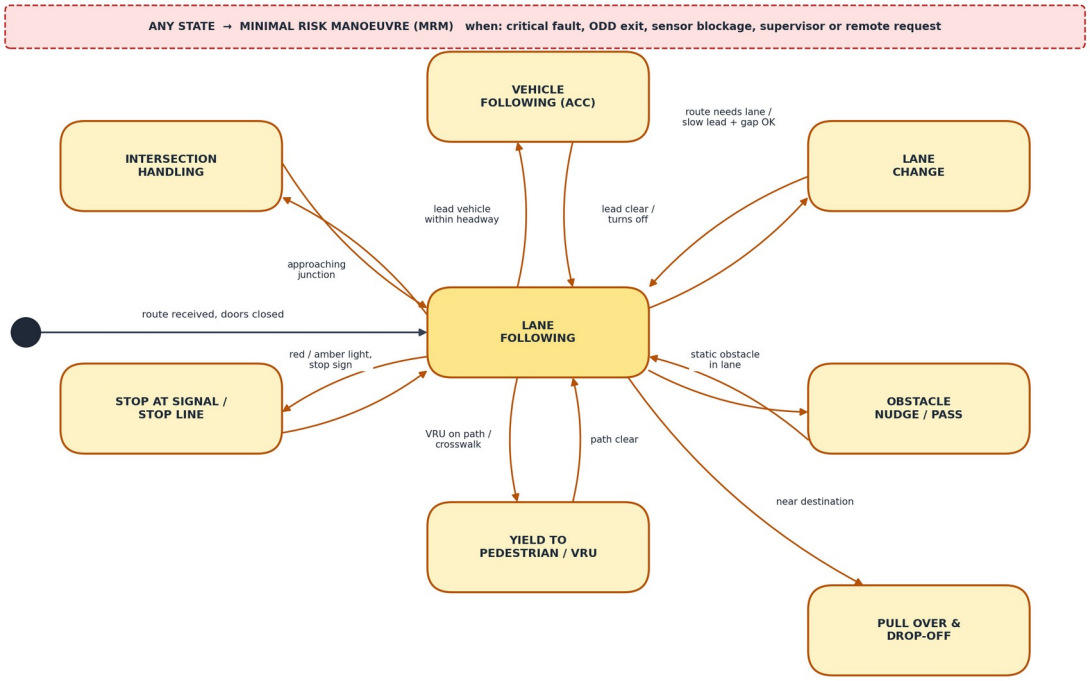


Figure 5: Behaviour planner state machine

Table 8: Behaviour planner states

State	Entry condition	Output to motion planner
Lane following	Default state, route continues in current lane	Keep lane centre, speed = min(limit, curve limit, comfort)
Vehicle following	Lead vehicle in lane within desired gap + margin	Adaptive time-gap following: $g_{des} = g_0 + h \cdot v$
Lane change	Route requires another lane or slow lead, and gap acceptance met	Target lane, lateral transition time 4-6 s
Obstacle nudge / pass	Static obstacle partly blocking the lane, oncoming lane clear	Lateral offset path, reduced speed
Intersection handling	Approaching junction on route	Creep and yield logic, protected / unprotected turn rules
Stop at signal / stop line	Red or amber light that cannot be cleared, stop sign	Stop point at stop line
Yield to pedestrian / VRU	Predicted path of a pedestrian or cyclist conflicts with ego path	Stop point before conflict zone
Pull over & drop-off	Within 45 m of destination	Stop point at kerbside drop-off location
Minimal risk manoeuvre	Critical fault, ODD exit, supervisor or remote request	Pull over if possible, else controlled in-lane stop

7.4 Motion (trajectory) planning

The motion planner generates a smooth, collision-free trajectory that satisfies the behaviour constraints. It works in the Frenet frame of the reference lane (s along the lane, d across it). A

lattice of candidate trajectories is sampled by varying end lateral offset, end speed and duration; each candidate is a quintic polynomial in d(t) and a quartic polynomial in s(t). Candidates that violate limits or collide with predicted obstacles are discarded, and the remaining ones are ranked by cost:

$$J = w_j \int_0^T j(t)^2 \, dt + w_T T + w_d d_T^2 + w_v (v_T - v_{ref})^2 + w_c C_{prox} \tag{5}$$

where j is jerk (comfort), T the manoeuvre time, d_T the final lateral offset, v_T the final speed and C_{prox} a proximity cost to other agents. The best candidate is then smoothed by a quadratic program (QP) that enforces curvature, acceleration and jerk limits.

7.5 Trajectory safety checker

A deterministic checker, written to a higher integrity level than the learned planner, verifies every trajectory before it reaches control. It checks collision against the occupancy grid and all tracks (including worst-case predictions), vehicle dynamic limits and map boundaries, and the Responsibility-Sensitive Safety (RSS) longitudinal distance:

$$d_{min} = \left[v_r \rho + \frac{1}{2} a_{acc} \rho^2 + \frac{(v_r + \rho a_{acc})^2}{2 b_{min}} - \frac{v_f^2}{2 b_{max}} \right]_+ \tag{6}$$

where v_r and v_f are the speeds of the rear (ego) and front vehicles, ρ the response time, a_{acc} the maximum acceleration during ρ, b_{min} the minimum braking the ego vehicle guarantees and b_{max} the maximum braking assumed for the front vehicle. If a trajectory fails the check, the previous safe trajectory is kept and a fallback stop is requested.

8. Control Module

The control module makes the vehicle follow the approved trajectory precisely and smoothly. It runs at 100 Hz, ten times faster than planning, and interpolates the trajectory between planner updates. Lateral (steering) and longitudinal (speed) control are designed separately, as shown in Figure 6, which is accurate enough within the ODD's speed and acceleration range.

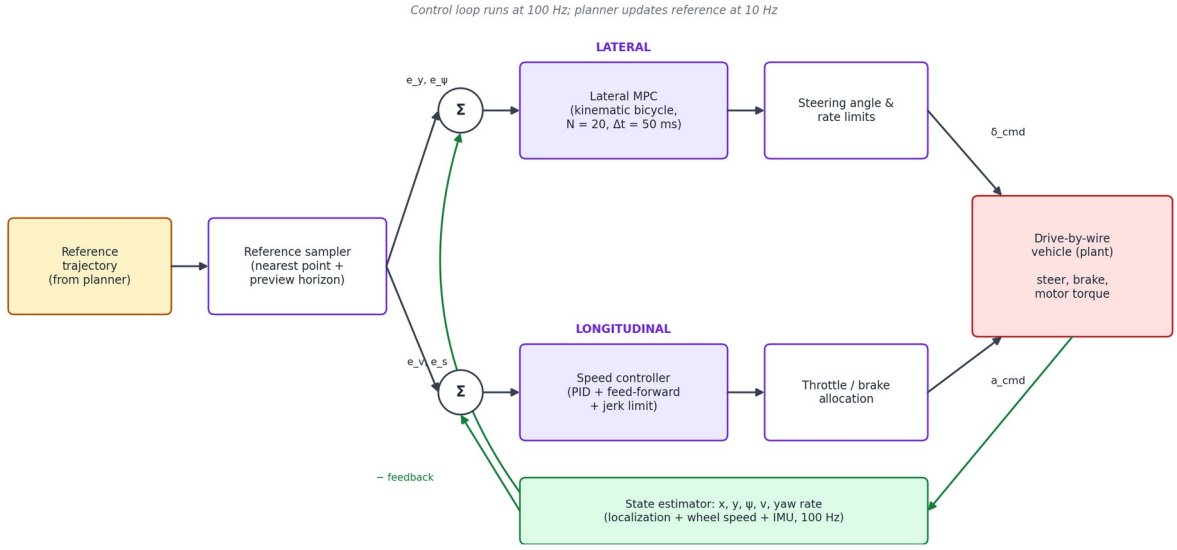


Figure 6: Lateral and longitudinal control loops

8.1 Vehicle model

The kinematic bicycle model, referenced at the rear axle with wheelbase L, describes the vehicle well at urban speeds and low lateral acceleration:

$$\dot{x} = v\cos\psi, \qquad \dot{y} = v\sin\psi, \qquad \dot{\psi} = \frac{v}{L}\tan\delta, \qquad \dot{v} = a \tag{7}$$

8.2 Lateral control

The primary lateral controller is a linear time-varying model predictive controller (MPC). At each step it linearises the model around the reference, predicts N = 20 steps of 50 ms (1 s horizon) and solves a small QP for the steering sequence that minimises lateral error e_y, heading error e_ψ and steering effort:

$$\min_{\delta_{0..N-1}} \sum_{k=0}^{N-1} \left(q_y e_{y,k}^2 + q_\psi e_{\psi,k}^2 + r_\delta \delta_k^2 + r_\Delta (\delta_k - \delta_{k-1})^2 \right) \quad \text{s.t.} \quad |\delta_k| \le \delta_{max}, \;\; |\delta_k - \delta_{k-1}| \le \Delta\delta_{max} \tag{8}$$

MPC handles actuator limits explicitly and anticipates curvature ahead. A geometric pure-pursuit controller runs in the safety computer as a simple, verifiable fallback, and is also used in the prototype (Section 11):

$$\delta = \arctan\left(\frac{2L\sin\alpha}{L_d}\right), \qquad L_d = k_0 + k_1 v \tag{9}$$

8.3 Longitudinal control

Speed is controlled with a PI controller plus feed-forward. The feed-forward term uses the planned acceleration, and when a stop point is active it uses the constant deceleration that brings the vehicle to rest exactly at the stop point. Acceleration and jerk limits keep the ride comfortable, and an allocation layer converts the demand into motor torque or brake pressure (with regenerative braking first).

$$a_{cmd} = a_{ff} + K_p\, e_v + K_i \int e_v\, dt, \qquad a_{ff,stop} = -\frac{v^2}{2\,(s_{stop} - s)} \tag{10}$$

8.4 Actuator interface

Table 9: Actuator command interface

Command	Range	Rate limit	Interface
Road-wheel steering angle	± 35°	≤ 30°/s (nominal)	CAN-FD to both steering ECUs
Longitudinal acceleration	−8 to +3 m/s²	Jerk ≤ 2.5 m/s³ nominal, higher in emergencies	CAN-FD to VCU and brake ECUs
Gear / parking brake	D, R, P / EPB on, off	n/a	CAN-FD
Body functions	Indicators, hazards, horn, doors	n/a	CAN-FD to body controller

9. Safety and Fallback Architecture

9.1 Redundancy and fault detection

Following ISO 26262, the ASIL D safety goals are met by decomposing them onto two independent channels: the primary computer (developed to a lower ASIL, with the learned components) and the safety computer (ASIL D, simple and verifiable). Faults are detected by heartbeats between all modules, plausibility checks between sensors (for example, a radar object that no camera or lidar confirms), range and rate checks on every signal, watchdogs on execution time, and comparison of the two computers' outputs at the actuator gateway.

9.2 Degradation modes and minimal risk manoeuvre

Figure 7 shows how the system moves between operating modes. A non-critical fault (for example one blocked camera, heavy rain, or a map mismatch) puts the vehicle into a **degraded** mode with lower speed and larger gaps. A critical fault or an imminent ODD exit starts a **minimal risk manoeuvre (MRM)**: the vehicle pulls over to the kerb or a safe stop zone if it still has enough capability, or otherwise brakes to a controlled stop in its lane. The **minimal risk condition** is reached when the vehicle is stationary with hazard lights on and the parking brake applied; the remote assistance centre is informed automatically.

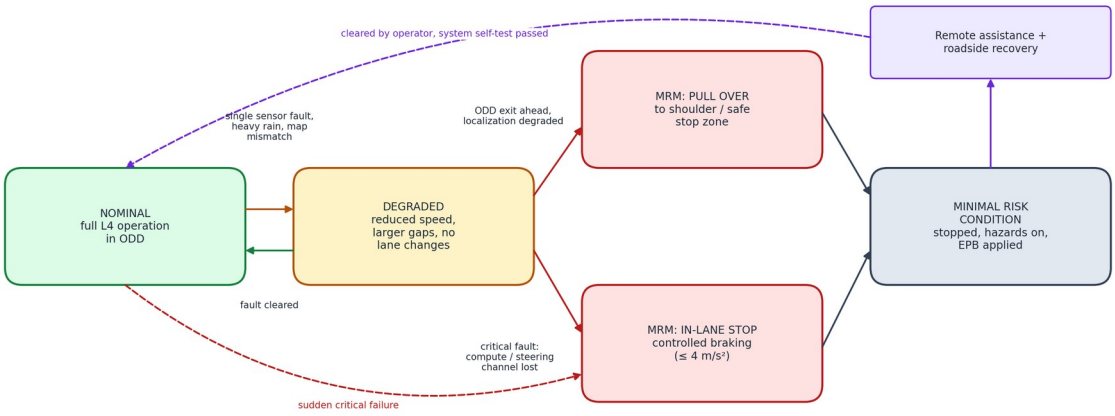


Figure 7: Degradation modes and transition to minimal risk condition

Table 10: Examples of faults and system responses

Fault example	Detection	Response
One camera blocked by dirt	Image quality monitor	Cleaning cycle; if persistent, degraded mode
Loss of long-range lidar	Heartbeat, point-count monitor	Degraded mode; pull over if localization confidence drops
Primary computer crash	Heartbeat timeout at safety computer (≤ 100 ms)	Safety computer takes control, in-lane or shoulder stop
Steering channel A failure	ECU self-diagnosis	Channel B continues; MRM pull over
Heavy rain beyond ODD	Wiper rate, rain sensor, perception confidence	MRM pull over at next safe stop

Fault example	Detection	Response
Loss of cellular link	Telematics watchdog	Complete current trip, no new trips

9.3 Remote assistance

Remote assistance operators support the vehicle in rare situations it cannot resolve alone, such as a road blocked by an unmapped obstruction or a police officer directing traffic. They provide high-level guidance (for example confirming a path or drawing a waypoint) which the vehicle's own planner and safety checker still verify. They never steer the vehicle in real time, because network latency makes that unsafe.

9.4 SOTIF and cybersecurity

- **SOTIF (ISO 21448)**: identifies hazards that occur without any fault, due to performance limits, such as misclassification of an unusual object. These are reduced by diverse sensing, the occupancy grid for unknown objects, conservative behaviour under uncertainty and scenario-based testing of triggering conditions.
- **Cybersecurity (ISO/SAE 21434, UN R155)**: secure boot, signed software updates (UN R156), encrypted and authenticated in-vehicle messages (SecOC) and a firewall between the telematics unit and the driving network.

10. Software Architecture and Timing

The software is built as a set of communicating processes on a service-oriented middleware. ROS 2 with DDS is used for development and research (as in Autoware); a production version would use AUTOSAR Adaptive on a safety-certified real-time operating system (such as QNX) for the primary computer and AUTOSAR Classic on the ASIL D microcontroller. Table 13 gives the timing budget from sensor capture to actuator command.

Table 11: End-to-end latency budget

Stage	Rate	Latency budget
Sensor capture and transfer	10-30 Hz	15 ms
Pre-processing and synchronisation	per frame	10 ms
Detection networks (on GPU / DLA)	10 Hz	40 ms
Fusion and tracking	10 Hz	10 ms
Localization (parallel)	100 Hz	5 ms
Prediction	10 Hz	15 ms
Behaviour and motion planning + safety check	10 Hz	30 ms
Control	100 Hz	2 ms
Total sensor-to-command		**≈ 122 ms (target ≤ 150 ms)**
Actuator response (brake pressure build-up)		50-100 ms (hardware)

11. Prototype Simulation and Results

11.1 Prototype description

To check that the module interfaces and the decision logic work together, a simplified version of the architecture was implemented in Python (folder **simulation/** of this project). It runs the perception, decision and control modules in a closed loop with a kinematic bicycle model of the vehicle. Table 14 shows how the prototype maps to the full design.

Table 12: Mapping between the design and the prototype

Design module	Prototype file	Simplification in prototype
World and scenario	av/world.py	Route of 430 m with a 30 m radius curve; scripted lead car, traffic light, pedestrian
Perception + tracking	av/perception.py	Ground truth with Gaussian noise, 80 m range gate, alpha-beta tracker (instead of IMM-EKF), objects in Frenet frame
Behaviour planner	av/decision.py	FSM with lane follow, vehicle follow, stop at signal, yield to VRU, pull over, MRM; curvature speed limit; conflict prediction for pedestrians
Control	av/control.py	Pure pursuit lateral (fallback controller of Section 8.2), PI + feed-forward longitudinal with jerk limit
Vehicle	av/vehicle.py	Kinematic bicycle, steering-rate and acceleration limits
Safety supervisor	run_sim.py / decision.py	Detects loss of perception output and commands MRM

11.2 Scenario A: nominal mission

The robotaxi starts from rest, follows a slower lead car (6 m/s) until it turns off, slows for a 30 m radius curve, stops at a red traffic light, yields to a pedestrian who starts crossing as it approaches, and finally pulls over at the destination 400 m along the route. Figure 8 shows the result: the behaviour state changes are shown as coloured bands behind the speed and acceleration plots.

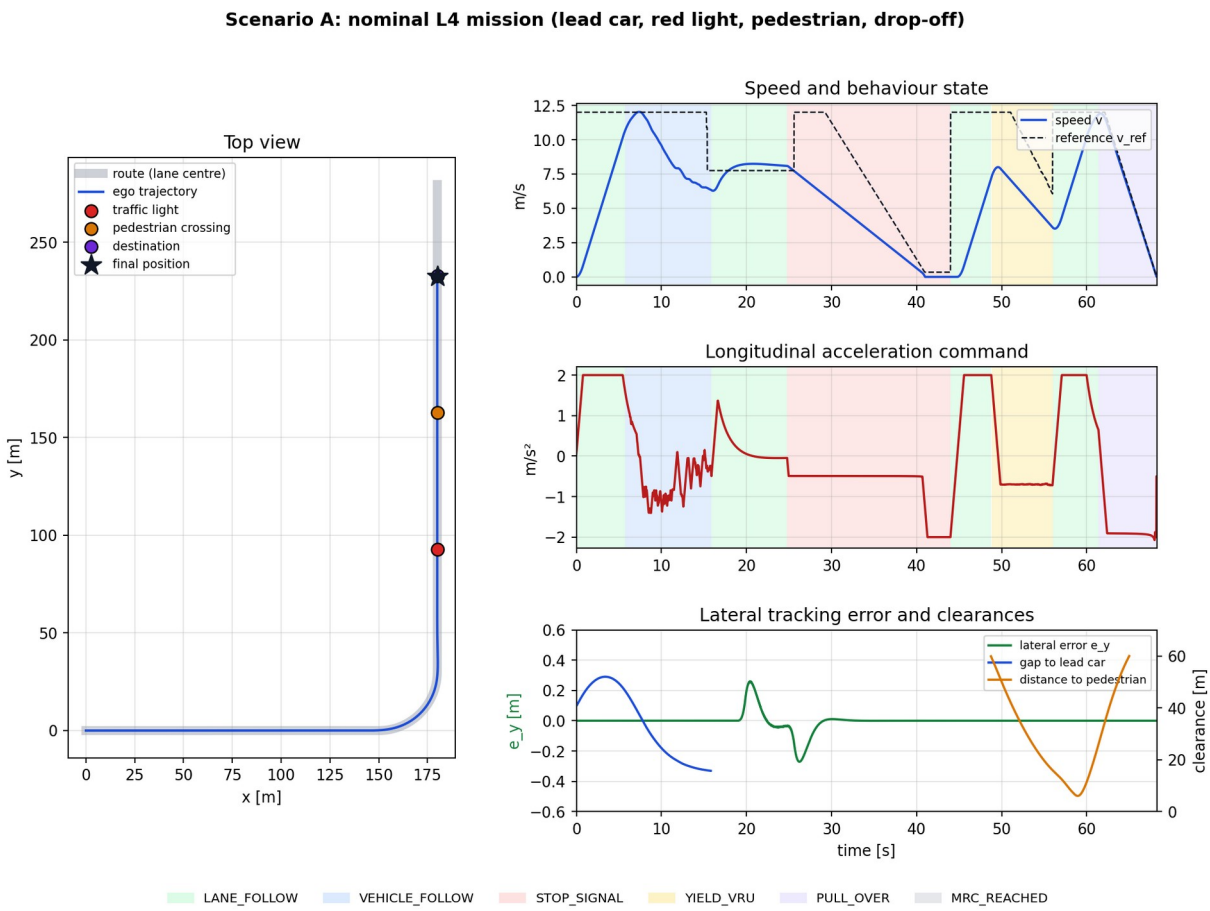


Figure 8: Scenario A: closed-loop simulation of a nominal mission

11.3 Scenario B: perception failure

The same mission is repeated, but the perception output is cut at t = 22 s while the vehicle is driving at 8.2 m/s just after the curve. The supervisor detects the missing world model at the next planning cycle and commands the minimal risk manoeuvre.

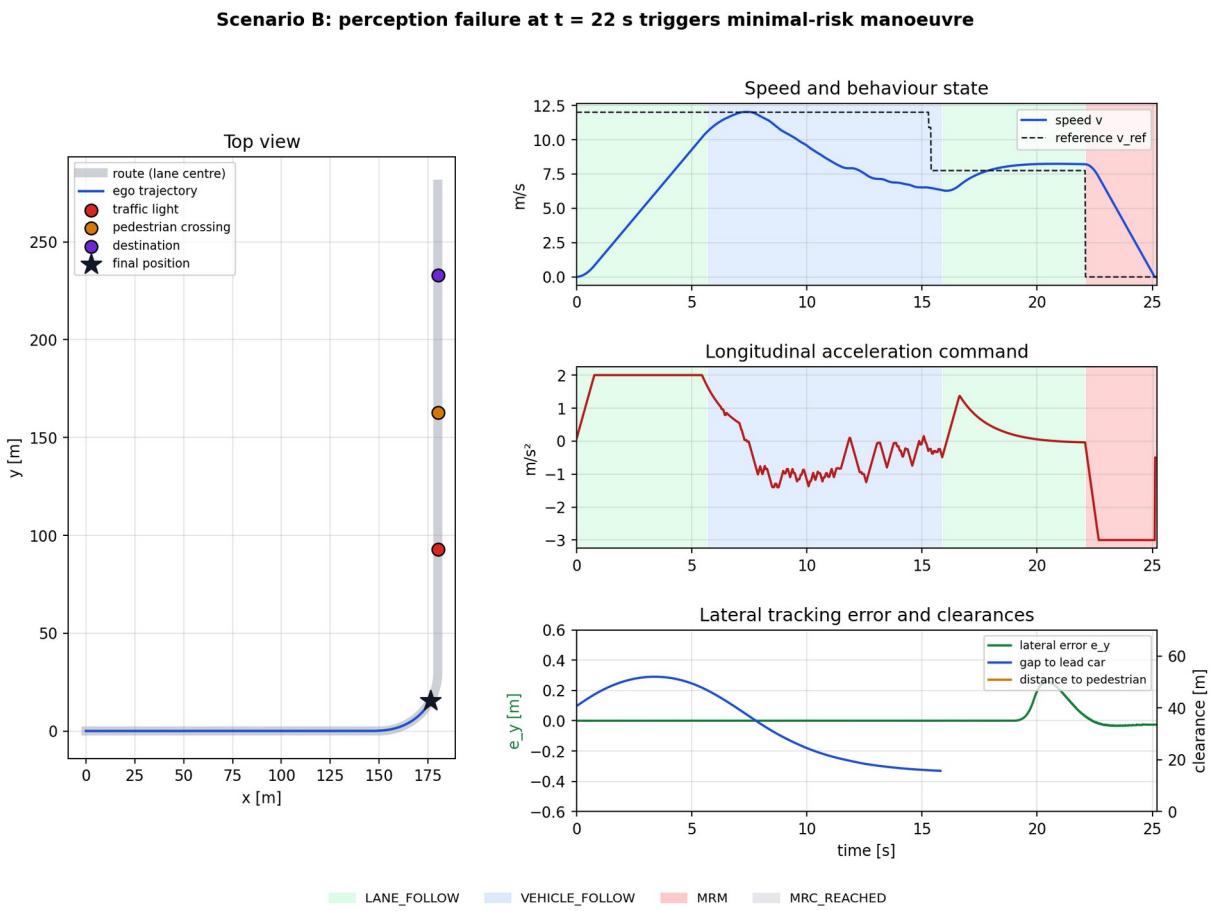


Figure 9: Scenario B: perception failure triggers a minimal risk manoeuvre

11.4 Results

Table 13: Simulation results

Metric	Scenario A (nominal)	Scenario B (fault)	Requirement
State sequence	Lane follow → vehicle follow → lane follow → stop at signal → lane follow → yield to VRU → lane follow → pull over → MRC	Lane follow → vehicle follow → lane follow → MRM → MRC	Correct mode for each event
Max lateral error	0.27 m (at curve entry)	0.26 m	≤ 0.30 m in curves
Minimum gap to lead car	15.7 m	15.7 m	≥ $g_0 + h \cdot v$
Stop at red light	0.33 m before stop line	n/a	Stop before line
Pedestrian interaction	Slowed to 3.6 m/s, minimum distance 6.0 m, resumed after pedestrian cleared	n/a	No conflict with VRU
Stop at destination	0.29 m before target	n/a	± 0.5 m
Maximum deceleration	2.1 m/s²	3.0 m/s² (MRM)	≤ 2 m/s² nominal, ≤ 4 m/s² MRM
Time from fault to MRC	n/a	3.2 s, stopped 14 m after fault	≤ 10 s

All requirements checked by the prototype were met. Nominal deceleration peaked at 2.1 m/s², very close to the 2 m/s² comfort target, when stopping at the destination. The lateral error peaked at the entry to the curve, which is expected for a pure-pursuit controller; the MPC of the full design would reduce this by anticipating the curvature. The prototype is deliberately simple and does not model sensor physics, multi-lane traffic or learned prediction, but it confirms that the interfaces, the state machine and the fallback path of the architecture behave as intended.

11.5 How to run

From the simulation folder, install the requirements (numpy, matplotlib) and run **python3 run_sim.py**. The script prints the metrics for both scenarios and saves the plots in simulation/results/. The fault time can be changed with **--fault 30**, for example.

12. Verification and Validation Strategy

1. **Unit and software-in-the-loop (SIL) tests** for each module against recorded data and synthetic inputs.
2. **Scenario-based simulation** with thousands of parameterised scenarios (OpenSCENARIO format) in simulators such as CARLA, including rare and critical cases (cut-ins, jaywalking, occluded pedestrians, sensor faults).
3. **Hardware-in-the-loop (HIL)** tests of the real computers and ECUs with simulated sensors and vehicle dynamics, including fault injection.
4. **Closed-course testing** of emergency manoeuvres, MRM and fault handling on a test track.
5. **Public road testing with safety drivers**, followed by **shadow mode**, in which the system plans in parallel with a human driver and disagreements are analysed.
6. **Driverless operation** in the ODD after a documented safety case (UL 4600) shows that the residual risk is acceptable, with continuous monitoring of safety metrics after launch.

13. Conclusion and Future Work

This project designed a complete functional architecture for a Level 4 urban robotaxi. The perception module fuses cameras, lidars and radars in a bird's-eye view and tracks every road user while localizing the vehicle to within centimetres on an HD map. The decision module predicts other road users' behaviour, plans the route, chooses the driving behaviour with a state machine and generates comfortable, collision-free trajectories that are independently checked against a safety envelope. The control module follows these trajectories with MPC and feed-forward control over redundant drive-by-wire actuators. Around these modules, an independent safety computer, degradation modes and a minimal risk manoeuvre make the vehicle able to handle its own failures, which is the defining property of Level 4. A closed-loop Python prototype confirmed that the module interfaces, behaviour logic and fallback path work as designed.

Future work includes replacing the prototype's simplified modules with the full algorithms (MPC, IMM tracking, learned prediction), testing in a high-fidelity simulator such as CARLA with simulated sensors, adapting the behaviour planner to Indian mixed-traffic conditions with

many two- and three-wheelers, and evaluating end-to-end learned planners within the same safety-checker framework.

References

[1] SAE International, "J3016_202104: Taxonomy and Definitions for Terms Related to Driving Automation Systems for On-Road Motor Vehicles," 2021.

[2] Waymo, "Meet the 6th-generation Waymo Driver," August 2024. https://waymo.com/blog/2024/08/meet-the-6th-generation-waymo-driver/

[3] The Autoware Foundation, "Autoware Documentation: Architecture overview." https://autowarefoundation.github.io/autoware-documentation/main/design/autoware-architecture/

[4] Baidu Apollo, "Apollo Auto open autonomous driving platform." https://github.com/ApolloAuto/apollo

[5] S. Shalev-Shwartz, S. Shammah and A. Shashua, "On a Formal Model of Safe and Scalable Self-driving Cars," arXiv:1708.06374, 2017.

[6] M. Werling, J. Ziegler, S. Kammel and S. Thrun, "Optimal trajectory generation for dynamic street scenarios in a Frenet frame," IEEE ICRA, 2010.

[7] T. Yin, X. Zhou and P. Krähenbühl, "Center-based 3D Object Detection and Tracking," IEEE CVPR, 2021.

[8] Z. Liu et al., "BEVFusion: Multi-Task Multi-Sensor Fusion with Unified Bird's-Eye View Representation," IEEE ICRA, 2023.

[9] J. Gao et al., "VectorNet: Encoding HD Maps and Agent Dynamics from Vectorized Representation," IEEE CVPR, 2020.

[10] R. Rajamani, Vehicle Dynamics and Control, 2nd ed., Springer, 2012.

[11] S. Thrun, W. Burgard and D. Fox, Probabilistic Robotics, MIT Press, 2005.

[12] ISO 26262:2018 Road vehicles: Functional safety; ISO 21448:2022 Safety of the intended functionality; ISO/SAE 21434:2021 Cybersecurity engineering.

[13] UL 4600, Standard for Safety for the Evaluation of Autonomous Products, 3rd ed., 2023.

[14] UNECE, UN Regulations No. 155 (Cyber security) and No. 156 (Software update), 2021.